

Distributed Systems: Consensus, Fault Tolerance & Replication

Course: CS402 Distributed Systems Architecture | License: CC-BY 4.0 Open Educational Resource

1. Fundamentals of Distributed Computing and System Models

A distributed system consists of autonomous computing entities (nodes) that communicate over a network by exchanging messages without shared physical memory or a global physical clock. System models categorize distributed environments into synchronous, asynchronous, and partially synchronous. In a synchronous model, there is a known upper bound on message propagation delay and relative processor speeds. In contrast, the asynchronous model makes no timing assumptions: message delays are unbounded, and processor execution rates are arbitrary.

2. Time, Causality, and Vector Clocks

Because physical clocks drift due to quartz crystal imperfection, Leslie Lamport introduced logical clocks to define a partial ordering of events based on the 'happened-before' relation (denoted by $\rightarrow$). If event a occurs before event b in the same process, then $a \rightarrow b$. If a is the sending of a message and b is its receipt, then $a \rightarrow b$. However, Lamport timestamps cannot detect causal independence: if $L(a) < L(b)$, it does not necessarily mean that a causally preceded b . Vector clocks resolve this limitation by maintaining a vector of logical timestamps representing each node's local view of progress. Event a causally preceded event b if and only if $V(a)[k] \leq V(b)[k]$ for all k and $V(a)[i] < V(b)[i]$ for at least one index i .

3. Distributed Consensus: Paxos and Raft

The consensus problem requires a set of nodes to agree on a single value or state machine transition despite node crashes and message drops. The FLP Impossibility Theorem (Fischer, Lynch, and Paterson, 1985) proved that deterministic asynchronous consensus is impossible if even a single node can suffer an unannounced fail-stop crash. Practical consensus algorithms therefore assume partial synchrony or randomized failure detectors.

Paxos, developed by Leslie Lamport, achieves consensus through two distinct phases: Phase 1 (Prepare/Promise) where a proposer seeks a quorum promise with a proposal number n , and Phase 2 (Accept/Accepted) where the proposer broadcasts an accepted value. Quorum intersection ensures that overlapping majorities guarantee safety and consistency across partition healing.

Raft decomposes consensus into three explicit subproblems: Leader Election, Log Replication, and Safety. Nodes exist in one of three states: Follower, Candidate, or Leader. Leaders maintain authority by sending periodic heartbeats (empty AppendEntries RPCs). If a follower's randomized election timer expires (typically 150ms to 300ms) without receiving a heartbeat, it transitions to candidate, increments the current term, and broadcasts RequestVote RPCs.

4. Two-Phase Commit (2PC) Protocol

Two-Phase Commit coordinates atomic transactions across distributed storage partitions. In the Prepare Phase, the coordinator sends a PREPARE message to all cohort participants; each participant executes the transaction locally up to the commit point, writes UNDO/REDO logs, and replies VOTE_COMMIT or VOTE_ABORT. In the Commit Phase, if all participants voted commit, the coordinator broadcasts GLOBAL_COMMIT; otherwise, it sends GLOBAL_ABORT. A major limitation of 2PC is that it is a blocking protocol: if the coordinator crashes after cohort nodes vote commit, participants remain blocked holding locks indefinitely.

5. Distributed Deadlock Detection and Handling

In distributed architectures, resources are distributed across distinct physical nodes. A distributed deadlock occurs when a global cycle of resource dependencies forms across multiple nodes, even though no single local node observes a cycle in its local Wait-For Graph (WFG). Distributed deadlock handling techniques fall into centralized, distributed, and hierarchical approaches. In centralized detection, nodes forward local WFG updates to a designated central deadlock detector. A major issue is phantom deadlocks, where network communication latency causes the detector to observe an obsolete cycle that has already resolved.

The Chandy-Misra-Haas edge-chasing algorithm detects distributed deadlocks by propagating probe messages (initiator, sender, receiver) along the edges of the distributed WFG. If a probe message returns to its original initiator process, a global deadlock cycle exists, triggering victim selection and process rollback.

6. Byzantine Fault Tolerance and Blockchain

Byzantine faults refer to arbitrary or malicious node behaviors, including forging messages, sending conflicting state updates to peers, or silently dropping transactions. Lamport, Shostak, and Pease demonstrated that achieving Byzantine agreement in a synchronous system with m faulty nodes requires at least $3m + 1$ total nodes and $m + 1$ communication rounds.

Practical Byzantine Fault Tolerance (PBFT) operates efficiently in asynchronous networks using three phases: Pre-Prepare, Prepare, and Commit, tolerating up to f malicious nodes given $3f + 1$ total replicas.